Influencer Finder

A complete manual for running YouTube creator discovery on your own machine — written for someone who has never opened a terminal.

RUNS ON	SETUP TIME	ACCOUNTS NEEDED	API KEYS
macOS & Windows	About 10 minutes	None	None

Everything in this manual happens on your own computer. There is no server to sign in to, no subscription and no key to paste. The app keeps its database in a folder next to itself and talks to nobody but YouTube.

CONTENTS

1 What this is, and what it never does 2

2 The one thing you have to install 4

3 Setting up on a Mac 5

4 Setting up on Windows 7

5 Starting it, every day after the first 9

6 A tour of the screen 11

7 Your first search 12

8 What the filters actually do 14

9 Reading the results 16

10 The other five tabs 18

11 When a search finds nothing 21

12 Turning the volume up 22

13 When something goes wrong 24

14 Moving it to another machine 26

15 Glossary 27

If you only read one page. Chapter 3 or 4 gets it installed, chapter 5 is the three lines you will type every day afterwards, and chapter 7 walks you through a first search. The rest is there for the day you need it.

CHAPTER ONE

What this is, and what it never does

Before installing anything, it is worth knowing exactly what you are putting on your computer — and, just as usefully, what it cannot do.

Influencer Finder is a tool for finding YouTube creators worth sponsoring, and for finding a way to contact them. You type in a few keywords — the sort of thing a viewer would search for — and it goes looking for the channels behind those videos. For each one it opens the channel's own About page, reads the real subscriber count, the country, the links, and any published business email, then measures how the channel is actually performing: how often it uploads, how many views it really gets, whether it is growing or drifting.

What comes back is a ranked table you can sort, filter, cull and export to Excel. Two further tools sit alongside it: one takes a creator you already like and finds others like them, and one reads a creator's last hundred uploads to work out who has already sponsored them and who is likely to next.

It runs on your machine, not ours

This is not a website you log in to. It is a program that runs on your own computer and opens a dashboard in your own browser at `http://localhost:8000`. The word *localhost* means precisely that — your machine, talking to itself. Nothing you type is sent anywhere except to YouTube, and the results are stored in a file inside the app's own folder.

The practical consequences are worth stating plainly:

- **There is no account and no password.** Nothing to sign up for, nothing to cancel.
- **There is no API key.** The tool does not use YouTube's official developer API, so there is no quota to run out of and no billing to watch.
- **Your list is yours.** The database lives in the `data` folder next to the app. Copy that folder and you have copied everything you have ever found.
- **If your laptop is closed, nothing is running.** Searches do not continue in the background on a server somewhere. Leave the window open until a run finishes.

What it will not do

Three honest limits, so that none of them arrive later as a surprise.

It cannot invent an email that does not exist. The tool reads the channel's About panel, its `mailto:` links, any linked contact or Impressum page, and the descriptions under recent videos. If a creator has published no business email anywhere, the `Email` column will be empty, and that is the true answer rather than a failure. In practice this is strongly tied to size: across a real database of channels, roughly **41%** of channels above 50,000 subscribers had a findable email, against about **25%** below 10,000. Small creators mostly do not publish one at all.

It will not guess a number it cannot verify. If a subscriber count cannot be tied to the channel's own page, the channel is dropped rather than estimated. You will occasionally see a run reject channels for

`subs unknown`; that is this rule working, not breaking.

It is not fast on your home connection, and should not be. On your own internet address the tool deliberately holds itself to a handful of requests a second. Push it harder and YouTube starts slowing you down, so the run takes *longer*, not shorter. Chapter 12 covers the proper way to go faster.

> **A note on the word "scraping".** The tool reads pages that are public to anyone with a browser, at a pace slower than a person clicking around would generate. It does not log in, does not touch private data, and does not attempt to get past anything designed to keep it out.

CHAPTER TWO

The one thing you have to install

Python. That is the entire list — no database, no Docker, nothing else. This chapter is only about checking whether you already have it.

Opening a terminal

A terminal is a window where you type commands instead of clicking. It looks intimidating and it is not; for this manual you will only ever paste in lines that are printed here and press return.

ON	DO THIS
macOS	Press `⌘` `Space`, type `Terminal`, press return.
Windows	Press the `Windows` key, type `PowerShell`, press return. Use PowerShell, not the older Command Prompt — one of the setup lines below only works in PowerShell.

Checking for Python

Paste this line and press return:

```
python3 --version
```

If it answers `Python 3.10.4` or any higher number, you are done with this chapter — skip to chapter 3 or 4. If it says *command not found* or *is not recognized*, you need to install it.

1. **Download the installer**
   Go to **python.org/downloads**. The big button offers the right version for whichever computer you are on.

2. **Run it, and take the defaults — with one exception**
   On **Windows**, the very first screen of the installer has a checkbox at the bottom reading **"Add python.exe to PATH"**. Tick it. It is easy to miss and it is the single most common reason the later commands fail. On **macOS** there is no such box; just click through.

3. **Close the terminal and open a new one**
   A terminal window only learns about programs installed before it opened. If you skip this it will keep insisting Python is not there.

> **Why `python3` and not `python`?** On a Mac there is no program called plain `python` — Apple ships only `python3`. On Windows both names usually work. Wherever this manual shows both, use whichever matches your machine; after step 2 of the setup, the plain name works everywhere.

CHAPTER THREE

Setting up on a Mac

Five steps, done once, ever. Most of the time is a download bar. If you are on Windows, skip to chapter 4 — it is the same shape with two different lines.

---

1 Get the terminal into the app's folder

The app lives in a folder called `influencer-finder`. In the terminal type `cd` — the letters c and d — then a **space**, and then **drag the folder out of Finder and drop it onto the terminal window**. It fills in the path for you, spaces and all. Press return.

```
cd /path/to/influencer-finder
```

To be sure you landed in the right place, type `ls` and press return. You should see `app.py`, `core`, `web` and `requirements.txt` among the names it lists. If you do not, you are one folder too high or too low.

2 Make a private space for the app's parts

This creates a hidden folder called `.venv` inside the project that holds this app's libraries only, so nothing it installs can disturb anything else on your Mac.

```
python3 -m venv .venv
source .venv/bin/activate
```

Your prompt now begins with `(.venv)`. That is how you know it worked, and it is the only signal you will get.

3 Install the parts

```
pip install -r requirements.txt
```

Four real dependencies and a helper: `curl_cffi` (makes the connection look like a genuine Chrome browser to YouTube), `fastapi` and `uvicorn` (the dashboard itself), `openpyxl` (Excel export) and `python-dotenv` (reads the settings file). A minute or two of scrolling text; a wall of green is normal.

4 Check it before you trust it

```
python app.py --check
```

This prints a tick or a cross against every dependency, then tells you which connection mode you are in, where the database will live, and where exported files will land. Four ticks means you are ready. It runs entirely offline — it does not touch YouTube — so it is safe to run any time you want reassurance.

5 Start it

```
python app.py
```

Your browser opens at `http://localhost:8000`. If it does not open by itself, type that address into the browser yourself.

> **Leave the terminal window open.** That window *is* the app. Close it and the dashboard goes dead. To stop the app deliberately, click the terminal window and press Control C.

If step 2 or 3 complains

The one error worth naming in advance is `externally-managed-environment`, which recent versions of macOS produce if you try to install without the `.venv`. It is not a fault — it is macOS telling you to do exactly what step 2 does. Run step 2, then step 3 again.

CHAPTER FOUR

Setting up on Windows

The same five steps. Two lines differ, and two errors happen on Windows and nowhere else — both are named below with their fixes.

USE POWERSHELL, NOT COMMAND PROMPT

1 Get PowerShell into the app's folder

Open the `influencer-finder` folder in File Explorer, click the address bar at the top so the path highlights, and copy it. Then in PowerShell type `cd`, a space, and paste it **inside double quotes** — the quotes matter if the path has spaces in it.

```
cd "C:\path\to\influencer-finder"
```

Type `dir` and press return to check. You should see `app.py`, `core`, `web` and `requirements.txt`.

2 Make a private space for the app's parts

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Your prompt now begins with `(.venv)`. If instead you get **"running scripts is disabled on this system"**, see the box below — it takes one line to fix.

3 Install the parts

```
pip install -r requirements.txt
```

Same five packages as on a Mac: `curl_cffi`, `fastapi`, `uvicorn`, `openpyxl` and `python-dotenv`. A minute or two.

4 Check it before you trust it

```
python app.py --check
```

Ticks against every dependency, then your connection mode and the folders it will use. All ticks means you are ready. It runs offline.

5 Start it

```
python app.py
```

Your browser opens at `http://localhost:8000`. Windows may show a firewall prompt the first time; **Allow** is safe — it is asking about your own computer talking to itself, not about anything reaching in from outside.

"Running scripts is disabled on this system."
Windows blocks locally written scripts by default, which includes the line that activates the `.venv`. Allow them for your own account only — this does not lower anything for anyone else on the machine:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Answer `Y`, then run the activate line from step 2 again.

"python is not recognized as the name of a cmdlet." The **"Add python.exe to PATH"** box was not ticked during installation. Re-run the Python installer, choose **Modify**, tick it, finish, then close PowerShell and open a fresh window.

Leave the PowerShell window open. That window is the app. To stop it deliberately, click it and press `Control` `C`.

CHAPTER FIVE

Starting it, every day after the first

Chapters 3 and 4 happen once per computer. From then on it is three lines — or one double-click, if you set that up now.

The three lines

- MACOS

```
cd /path/to/influencer-finder
source .venv/bin/activate
python app.py
```

- WINDOWS

```
cd "C:\path\to\influencer-finder"
.venv\Scripts\Activate.ps1
python app.py
```

> **The middle line is the one people forget.** The private space has to be re-activated in every new terminal window. If you skip it, `python app.py` answers `ModuleNotFoundError: No module named 'fastapi'`. That message never means anything is broken — it means only "you forgot to activate".

Or make it a double-click

If you would rather not remember any of it, save one small file inside the project folder and launch the app by double-clicking it from now on.

- MACOS — SAVE AS `START.COMMAND`

```
#!/bin/bash
cd "$(dirname "$0")"
source .venv/bin/activate
python app.py
```

Then run this once, to give the file permission to launch:

```
chmod +x start.command
```

- **WINDOWS — SAVE AS `START.BAT`**

```
@echo off
cd /d "%~dp0"
call .venv\Scripts\activate.bat
python app.py
```

Flags worth knowing

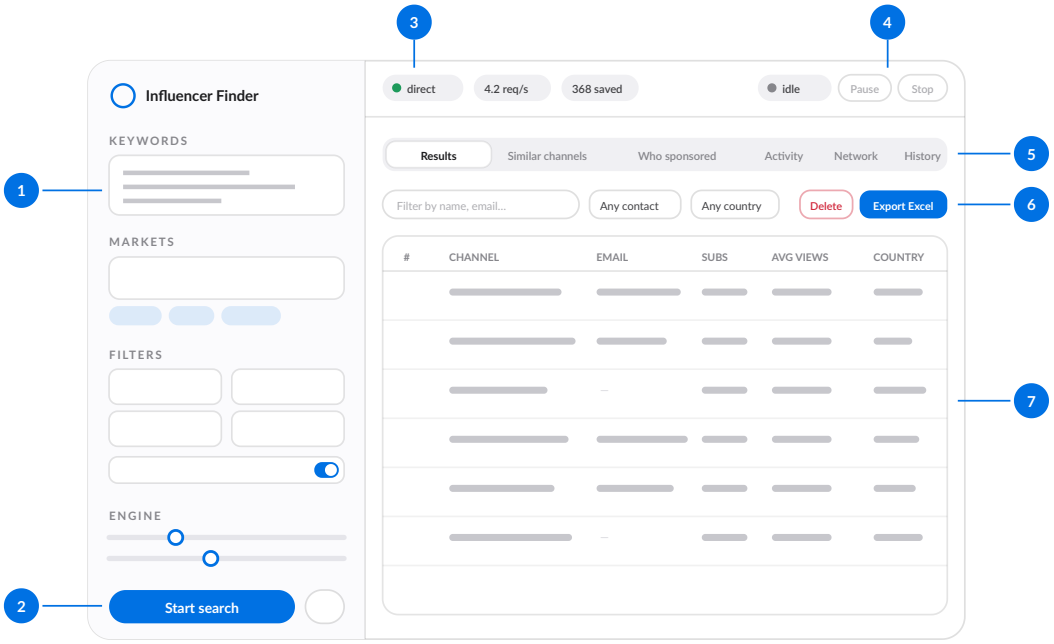
A flag is a word added after the command that changes how it starts.

LINE	WHAT IT DOES
`python app.py --port 9000`	Runs the dashboard somewhere else. Use this when something already has port 8000.
`python app.py --no-browser`	Starts without opening a browser tab.
`python app.py --host 0.0.0.0`	Lets another machine on your own network open the dashboard. Only do this on a network you trust.
`python app.py --check`	Verifies the setup and exits. Offline, harmless, run it whenever.
`python app.py --diagnose @mkbhd`	Walks one channel through every stage of the pipeline and prints what each one returned. This is the tool you reach for when a search finds nothing and you want to know whether the machinery or your settings are at fault.

CHAPTER SIX

A tour of the screen

The dashboard has three regions and never changes shape: you set things up on the left, you watch the state along the top, and you read results in the middle.



The dashboard, once a search has finished. Nothing in this layout moves or reorganises itself; the same seven things are always in the same seven places.

1. **The search panel.** Everything you choose *before* pressing start: keywords, markets, filters, engine speed. Press ⌘ \ to fold it away and give the table the whole window.
2. **Start search** and, beside it, **Save** — which remembers your current settings in this browser so they are waiting next time.
3. **Status pills.** Your connection mode, the live request rate, and how many channels are in the database altogether.
4. **Pause** and **Stop**. Grey until a run is going. Pause holds the run where it is; Stop ends it and keeps everything found so far.
5. **The six tabs.** Chapter 10 takes each one in turn. The number beside **Results** is how many channels are currently in the table.
6. **The results toolbar.** Filter the list down, tick what you want, **Delete** what you don't, then **Export Excel**.
7. **The table.** Rows arrive live as they are found. Click any row to open it and see the reasoning behind its rank.

A dash in the Email column is an answer. It means the tool looked in all four places a business email can live and the creator had published none. It is not a row that failed to load.

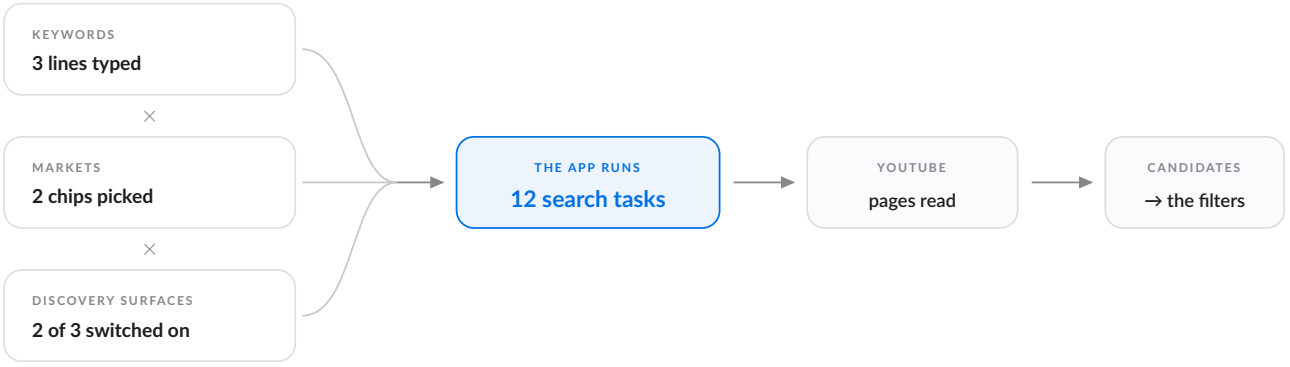
CHAPTER SEVEN

Your first search

Make it deliberately small. The point of the first run is to learn what the tool feels like on your connection, not to build a list.

What a search is made of

Before you press anything, it helps to know what the tool is about to do with what you typed. Your keywords, your markets and your discovery surfaces are multiplied together into a list of separate searches — and the hint line under the keyword box tells you the total before you commit.



Three lines, two markets and two surfaces is twelve searches, not three. The hint under the keyword box prints this total live, in the form `3 keyword(s) × 2 market(s) × 2 surface(s) = 12 search tasks`. It is the single best predictor of how long a run will take, and the reason to start with one of each.

> **Markets steer the search. They have never filtered it.** Picking **US** and **GB** tells YouTube where to search *from* — but YouTube ranks by relevance, not by region, so a US search happily returns the world. The control that decides whether a creator's own published country has to match is **Country scope**, directly underneath. See chapter 8.

The run itself

1. **Type one keyword**

   In the **Keywords** box, one per line — start with exactly one, something a viewer would actually type, like `ai tools`. Not a category, a search.

2. **Pick one market**

   Click a single country chip under **Markets**. Leave **Country scope** on its default, **Only the markets above**.

3. **Leave every filter where it is**

   Min subs at 50,000 in particular. This matters more than it looks: below about 10,000 subscribers most creators publish no business email at all, so a low floor fills the table with rows whose **Email** column can only ever be a dash.

4 **Leave the engine alone**

`Depth per keyword` at 100 and `Workers` at 4. Chapter 12 explains when and how to raise these — and why raising `Workers` on your home connection makes runs slower rather than faster.

5 **Press** `Start search`

A thin progress bar appears under the top bar and a single live line describes what is happening. Rows arrive in `Results` as they are found — you do not have to wait for the run to end before sorting, filtering or exporting.

> **Expect single digits, and be pleased with them.** One keyword, one market, depth 100 and a 50k floor is a narrow net by design. Somewhere between two and fifteen keepers is a healthy first run. Nothing at all is also informative rather than broken — chapter 11 is about exactly that.

Where the results go

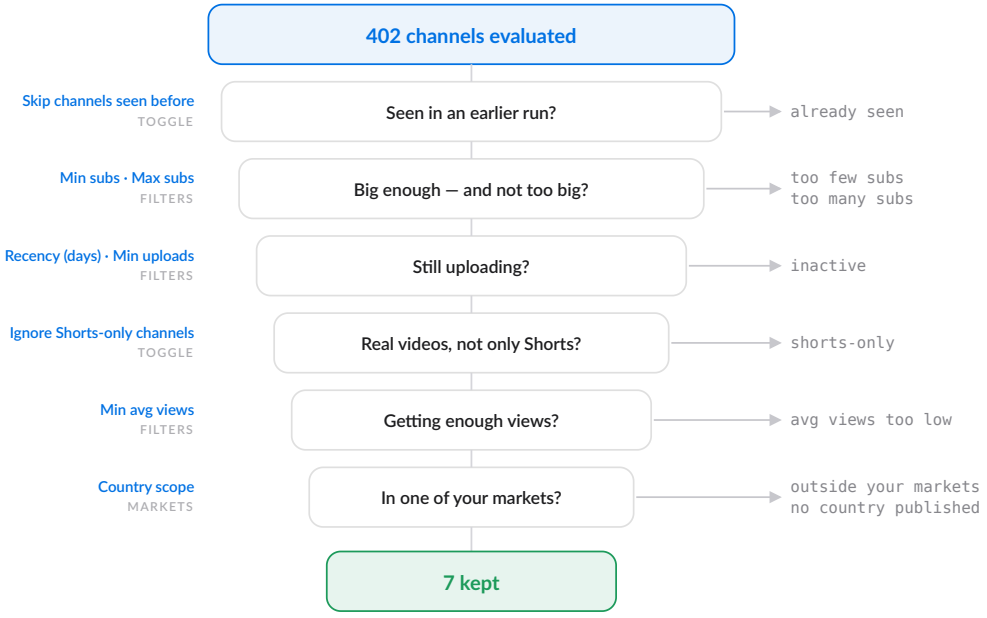
Everything a run keeps is written into the database immediately, so it survives closing the browser, closing the app and restarting your computer. Each channel is stored once: if a later search finds the same creator, it updates the existing entry rather than adding a duplicate.

The tool also keeps a private **seen list** — every channel it has ever evaluated, including the ones it rejected. This is what `Skip channels seen before` consults, and it is why your second run on a similar keyword is much faster than your first. It is also, as chapter 11 explains, the most common reason a later run appears to find nothing.

CHAPTER EIGHT

What the filters actually do

Every candidate channel is put through the same gauntlet, in the same order. Each stage rejects for one named reason, and every reason maps back to exactly one control you can change.



Every kept channel is then verified against its own About page — subscribers, country, links and email.
A number that cannot be tied to the channel itself is dropped, never guessed.

The gauntlet, in order. On the left, the control that governs each stage; on the right, the exact words the app will use when it tells you why channels were rejected. Those words appear in the run line while a search is going, and on the card afterwards in `History`.

The controls, one by one

CONTROL	WHAT IT MEANS, AND WHAT TO DO WITH IT
Min subs	The subscriber floor. Default 50,000. The single most consequential setting in the panel, because it is also the strongest predictor of whether an email exists at all. Halve it if a niche is genuinely small; expect more blanks in **Email** when you do.
Max subs	A ceiling, for when you want the reachable middle rather than the household names. `0` means no ceiling.
Recency (days)	How far back an upload still counts as recent. Default 60. Widen it to 90 in slow-moving niches.
Min uploads	How many uploads a channel needs *inside* that window to count as active. Default 4. This and the line above work as a pair: four uploads in 60 days is stricter than it sounds.

CONTROL	WHAT IT MEANS, AND WHAT TO DO WITH IT
Min avg views	Optional floor on the average view count. Leave it at `0` until you have a reason; it is the easiest way to silently empty a run.
Ignore Shorts-only channels	On by default. Channels that publish almost nothing but Shorts are rejected. Turn it off for a niche where Shorts genuinely are the format.
Skip channels seen before	On by default. Anything already evaluated by any earlier run is passed over without being fetched again. This is what makes repeat runs quick — and what makes a fourth run on the same keyword return nothing.
Country scope	Three settings. **Anywhere in the world** keeps everyone. **Only the markets above** keeps creators in your markets, plus those who published no country at all. **Only the markets above, strictly** also drops the blanks. Start with the middle one.
Read video descriptions for emails	On by default. Adds a pass over recent video descriptions on top of the About page. Slower, and worth it — descriptions are where a good share of business emails actually live.
Find sponsor brands	On by default. Reads the same descriptions for sponsorships, and adds a **Sponsors** column to the table and the export.

CHAPTER NINE

Reading the results

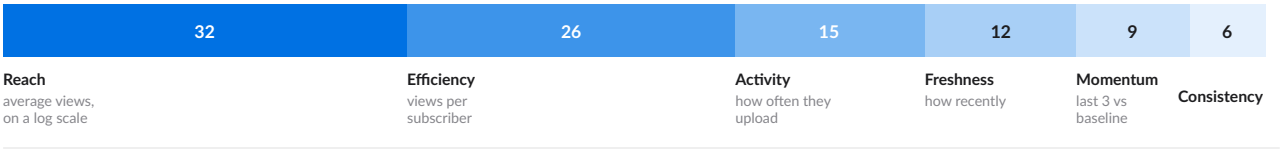
Eight columns, a rank, and a drawer underneath every row that shows the working behind that rank.

The columns

COLUMN	WHAT IT IS
#	The rank. By default the table is sorted by opportunity score, best first. Click any other heading to sort by it instead; click again to reverse.
Channel	Name and `@handle`, linked to the channel. This column stays put when you scroll sideways.
Email	The business email, if the creator published one. A dash means all four places were empty.
Subscribers	Read from the channel's own About page, never estimated from a search listing.
Average views	A *trimmed* mean of recent uploads — the highest and lowest are discarded first, so one viral video cannot make an ordinary channel look extraordinary.
Country	The country the creator published on their own About page. Blank means they published none, which is common and not an error.
First name / **Last name**	The person behind the channel where it can be established, so an outreach email can open with a name rather than a channel.
Sponsors	Brands found in recent descriptions. The column that most often decides whether an email gets written today or next quarter.

The score, and what it rewards

The number in the **#** column is a rank derived from a 0–100 opportunity score. It deliberately favours *efficient* mid-size channels over big lazy ones: a 200,000-subscriber channel pulling 150,000 views a video is a far better sponsorship buy than a two-million-subscriber channel pulling 40,000.



What the score is made of. Reach and efficiency together are well over half of it, and efficiency alone outweighs everything about cadence — which is the whole argument of the ranking in one picture.

When views cannot be measured. Occasionally YouTube reports no view counts at all for a channel. Rather than scoring it zero and burying a possibly excellent creator at the bottom next to genuine junk, the tool scores it on cadence and freshness alone and caps the result at 55 — mid-table, with the reasoning visible in the drawer.

The drawer

Click any row and it opens. Inside are six measurements — `Score`, `Momentum`, `Views / sub`, `Uploads`, `Last upload` and `Consistency` — the sponsorships found and the upload each was found in, the ten most recent videos with their view counts, and the channel's published links. This is where you check the tool's homework before writing to anyone.

Culling and exporting

- **Filter first.** The search box matches name, handle, email and country. The two dropdowns narrow by contact (`Has an email` / `No email`) and by country.
- **Then tick and delete.** Click the tick column on any row, or use `Select all`. `Delete` stays grey until something is ticked, so it can never be the button you hit by accident — and a toast with `Undo` appears afterwards.
- **Then export.** `Export Excel` writes an `.xlsx` into the `data` folder and lists it under `History`.

Read the counter beside `Select all` **.** With nothing ticked it does not say "0 selected" — it reports how much of the list you can actually write to, in the form `148 of 361 have an email · 41%` . That is the one fact about a finished list you cannot get by looking at it.

CHAPTER TEN

The other five tabs

Results is where a search lands. The value, once you have a list, is mostly in the two tabs beside it.

Similar channels

Start from one creator you already like rather than from a keyword. Paste a channel URL, an `@handle` or a channel ID and press **Find similar**. The tool reads that channel, then follows YouTube's own recommendation rail to find its neighbourhood.

What comes back is a table, ranked by **Match** — which weighs co-viewing first, then how comparable the neighbours are in size and country. The last column, **Why it ranked here**, spells out the reasoning for each row in words. Tick the ones you want and press **Add to results**; they join the main table and the export like anything else.

> **This is the highest-yield habit in the tool.** One good find from a keyword search becomes a shortlist of fifteen. It is usually a better use of ten minutes than another keyword.

Who sponsored

Everything one creator has been paid to say. Paste a creator and press **Build report**. The tool lists their uploads newest first and reads the description of each of the most recent hundred, pulling out every sponsorship it can find — which brands, how many videos each, when the relationship started, and whether it is still running.

You get a row of headline figures first (**brands sponsored**, **still running**, **came back**, **of uploads sponsored**, and so on), then the full sponsorship history:

COLUMN	WHAT IT TELLS YOU
Brand	The company, linked to its site. A filled dot means the sponsorship ran recently.
Videos	How many placements that brand bought from this creator.
First / **Last**	When the relationship started and when it was last seen. A wide gap with a recent **Last** is a renewing sponsor.
Also works with	How many *other* creators in your database that brand has sponsored, and their median size. This is the number that turns a name into a decision — a brand with a roster has a programme, a rate card and somebody whose job it is to answer.
Proof	Links to the actual uploads the sponsorship was read from. Always check one before you act on it.

Underneath sits **Brands to pitch next** — companies ranked by how likely they are to buy this creator, scored against the brand ledger the tool builds as you search. This ranking gets sharper as the ledger fills. If you ran searches before this feature existed, the empty state offers a link to fold those past runs in.

> **About a minute the first time, near-instant afterwards.** Every description the tool reads is cached, so a second report on the same creator — or any later question about the same niche — costs almost nothing.

Activity

The running commentary: every channel accepted, every rejection and its reason, every rate-limit the engine backed off from. Follow the newest line keeps it scrolled to the bottom; untick it to read back without the view jumping. This is the first place to look when a run behaves oddly while it is still going.

Network

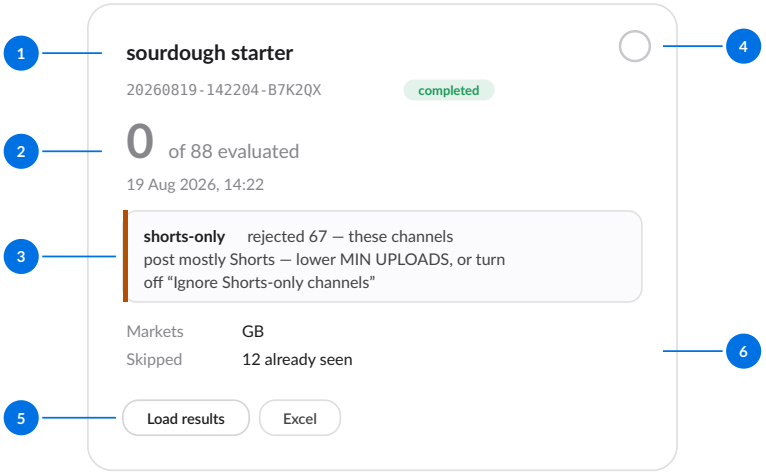
Where connections are configured, without editing any file. Three modes:

- Direct — your own internet connection. Correct until you have a reason otherwise.
- Rotating gateway — one address from a provider that hands out a fresh exit IP. Paste the gateway URL and set how long each worker keeps one IP.
- Proxy list — your own proxies, one per line. Unhealthy ones are benched automatically and let back in when they recover.

Press Save & apply to switch, and Test rotation to prove the exit IP really is changing before you trust a long run to it. Health cards for each proxy appear underneath.

History

One card per search you have run, plus the export folder. The card does not simply tally what happened — where a run kept nothing, it says why.



A run that kept nothing, explaining itself. The keywords are the headline and the run ID is demoted to a caption, because you look for a run by what you searched for, never by its number.

1. **What you searched for.** Up to six keywords, then an ellipsis.
2. **Kept, out of evaluated.** Grey when the answer is zero.
3. **The post-mortem.** The biggest single cause, the count, and what to change. Amber for a filter; blue when the cause was `already seen`.
4. **The selection tick.** Click anywhere on the card except the two buttons.
5. `Load results` puts that run's channels back in the table. `Excel` exports just that run.
6. **Contactable and Skipped.** Email coverage for the run, and how many channels were passed over as already seen.

Clearing the log

Cards accumulate. Select the ones you no longer want — or press `Select the empty ones`, which ticks every run that kept nothing in one go — and press `Delete`.

> **Deleting a card deletes only the card.**
> A run log and a lead list are different assets on different clocks. Removing a run removes the diary entry; every channel it found stays in `Results`, in the database and in your exports, and the seen list is untouched. The toast says so, and offers `Undo` for nine seconds.

> **"N struck out since".** Occasionally a card shows fewer found than it originally kept. That is not lost data — it means you later deleted some of those channels from `Results`. The card counts live rows, so it can never promise more than `Load results` will deliver.

CHAPTER ELEVEN

When a search finds nothing

This is almost never a breakage. It is a filter, and the app already knows which one — the trick is knowing where it says so.

Two places tell you. While a run is going, the single live line under the top bar names the biggest cause as it happens. Afterwards, the card in **History** carries the same verdict permanently.

Every rejection reason maps to one control. Find the reason you were given in the left column and change the thing in the right.

IF IT SAYS	IT MEANS, AND WHAT TO CHANGE
`already seen`	Every channel had been found by a previous run. Turn off **Skip channels seen before** for this run, or move to a new keyword. **This is the most common cause by a distance** — it is the price of the speed that same setting buys you.
`shorts-only`	These channels post mostly Shorts. Lower **Min uploads**, or turn off **Ignore Shorts-only channels**.
`too few subs`	**Min subs** is above almost everyone this keyword finds. Try halving it.
`too many subs`	**Max subs** is cutting the results off. Raise it, or set it to `0`.
`inactive`	Nobody cleared **Min uploads** inside **Recency (days)**. Widen the window to 90, or drop the upload count.
`avg views too low`	**Min avg views** is too high for this niche. Lower it or clear it.
`outside your markets`	These creators are real, they just live somewhere else. Add their country to **Markets**, or set **Country scope** to **Anywhere in the world**.
`no country published`	**Country scope** is on *strictly*, and these creators left the country field blank. Switch to **Only the markets above** to keep them.
`subs unknown`	No subscriber count could be tied to the channel itself, so it was dropped rather than guessed. Run `python app.py --diagnose <channel-url>`.
`no feed`	YouTube served no upload feed — usually rate limiting. Lower **Max requests / sec**, or add a proxy.

Proving the machinery is fine

If you would rather be certain the pipeline itself is healthy before touching any filter, point the self-check at a channel you *know* should pass:

```
python app.py --diagnose @mkbhd
```

It walks that one channel through every stage and prints what each returned. All ticks means the machinery is sound and the problem is your settings; a cross names the stage that broke.

CHAPTER TWELVE

Turning the volume up

Volume comes from connections, not from threads. Raising the wrong dial makes runs slower, and this chapter is mostly about which one that is.

The settings file

Defaults live in a file called `.env` in the project folder. It does not exist until you make it — copy the example that ships alongside it:

- MACOS

```
cp .env.example .env
```

- WINDOWS

```
copy .env.example .env
```

Then open `.env` in any text editor. Anything you set here becomes the starting value in the sidebar; anything you change in the sidebar applies to that run only.

SETTING	DOES WHAT	SENSIBLE VALUE
`WORKERS`	Channels evaluated at the same time	`4` on your own connection; `24` behind a gateway
`MAX_RPS`	Ceiling on requests per second across everything	`6` direct; higher only with proxies
`MIN_SUBSCRIBERS`	Subscriber floor	`50000`
`SEARCH_DEPTH`	Videos scanned per keyword	`100` to start, `500` + for real runs
`RECENT_DAYS`	How far back an upload still counts	`60`
`MIN_RECENT_VIDEOS`	Uploads needed inside that window	`4`
`COUNTRY_FILTER`	The Country scope default	`markets`
`PROXY_MODE`	`direct`, `gateway` or `list`	`direct` until you have proxies
`PORT`	Where the dashboard listens	`8000`

Do not raise `WORKERS` past about 4 on your own connection. YouTube starts throttling a single address that asks too fast, and the run gets *slower*. The app will warn you if you try to start a direct run with a high worker count, and the warning is worth heeding rather than clicking through.

Where speed actually comes from

Two dials, in this order.

- `Depth per keyword` **buys you reach.** Going from 100 to 500 means five times as many videos scanned per keyword, so five times as many candidate channels. It costs time linearly and carries no risk. This is the honest way to get a bigger list.
- **Proxies buy you parallelism.** A rotating gateway spreads the requests across many addresses, so `WORKERS` at 24 or 48 is comfortable where 4 was the limit. This is the only way to genuinely multiply throughput.

Setting up proxies

You do not need to edit `.env` for this. Open the `Network` tab, choose `Rotating gateway`, paste the URL your provider gave you, and press `Save & apply`. Then press `Test rotation` and confirm the exit address really changes between requests before committing a long run to it.

The setting that most decides whether proxies work well or badly is the **sticky session** — how long each worker keeps one exit address before rotating. The slider in the `Network` tab reads `Keep each worker on one IP for N requests`. The default, 25, is a good compromise: the traffic reads as several people browsing rather than one machine teleporting between continents. Setting it to `0` takes a brand-new address on every single request, which sounds safer and is usually worse.

CHAPTER THIRTEEN

When something goes wrong

Every error you are realistically going to meet, with what it actually means. Most of them mean "you forgot a line", not "something broke".

STARTING UP

MESSAGE	WHAT IT MEANS
`zsh: command not found: python`	macOS ships `python3`, not plain `python`. Use the `3` version, or activate the `.venv` — inside it the plain name works.
`ModuleNotFoundError: No module named 'fastapi'`	The `.venv` is not active in this terminal window. Run the activate line and try again. Nothing is broken.
`externally-managed-environment`	You tried to install without the `.venv`. Do step 2 of the setup, then step 3 again.
`running scripts is disabled on this system`	Windows only. Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, answer `Y`, then activate again.
`python is not recognized`	Windows only. The **Add python.exe to PATH** box was missed. Re-run the installer, choose **Modify**, tick it.
`Address already in use`	Something has port 8000 — quite possibly a copy of this app still running in another terminal window. Use `python app.py --port 9000`.
`curl_cffi` won't install	It ships pre-built for common platforms; if pip is compiling it from source you are on an unusual Python version. Python 3.11 or 3.12 is the safest ground.

WHILE IT IS RUNNING

The page at localhost:8000 will not load. Check the terminal window is still open and has not printed an error. If you started with `--port 9000`, the address changed to match.

The search runs but finds nothing. Almost always the filters. Chapter 11 is entirely about this, and the app has already worked out which filter it was — look at the card in History.

The run has slowed to a crawl. Open Activity. If it is full of rate-limit lines, YouTube is throttling you. Lower Max requests / sec and Workers, or add proxies. The engine already backs off by itself when it sees this, so a run that slows down is the safety working.

Most emails are blank. Expected, and worth internalising rather than troubleshooting: below about 10,000 subscribers most creators publish no business email anywhere, so there is nothing to read. Raise Min subs and re-run before concluding anything is wrong. Even at the top of the range the ceiling is real — roughly four in ten channels above 50,000 subscribers have a findable address.

Countries look wrong or blank. The country shown is the one the creator published on their own About page, not one inferred from language or search region. Blank means they published none. If you are getting creators from the wrong side of the world, the control you want is Country scope, not Markets — see chapter 8.

> **The check that costs nothing.** `python app.py --check` runs offline and prints a tick or cross for every dependency plus the folders in use. When you are not sure whether the problem is the app or the network, run it first.

CHAPTER FOURTEEN

Moving it to another machine

Copy the folder, delete one thing, run the setup again. But know what you are carrying.

1. **Copy the whole `influencer-finder` folder**
   A USB stick, a cloud folder, anything.

2. **On the new machine, delete `.venv`**
   It contains paths baked in to the old computer and will not work on the new one. Deleting it is safe; it is rebuilt in the next step.

3. **Run the setup from step 2 onwards**
   Chapter 3 for a Mac, chapter 4 for Windows. Two commands and a wait.

What is in the folder

FOLDER OR FILE	WHAT IT HOLDS
`data/`	The database — every channel you have ever found, every brand ledger entry, every cached video description, and your Excel exports. Carry it and the new machine inherits all of it, including the caches that make repeat lookups instant. Leave it behind and you start clean.
`.env`	Your settings **and your proxy credentials**.
`proxies.txt`	Your proxy list, if you use list mode. Also credentials.
`.venv/`	The installed libraries. Machine-specific — always delete before moving.

Before you send this folder to anyone else. Delete `.env` and `proxies.txt` first, or you are sending them your proxy password. Consider whether you mean to hand over `data/` as well — it is your entire lead list.

Backing up

There is nothing clever to do. Copy the `data` folder somewhere safe while the app is not running. That single folder is the whole of your work.

CHAPTER FIFTEEN

Glossary

Every term below is one the tool actually puts on your screen or in its settings file, quoted exactly. Nothing here is invented for the manual.

ON THE SEARCH PANEL

Keywords
What a viewer would type into YouTube. One per line. Three lines means three separate searches, not one search for three words.

Markets
Country chips telling YouTube where to search *from*. They steer the search; on their own they have never filtered the results.

Country scope
The control that decides whether a creator's own published country has to match your markets. `Anywhere in the world` keeps everyone; `Only the markets above` keeps your markets plus creators who published no country; `Only the markets above, strictly` drops the blanks too.

Min subs Filters
Subscriber floor. Also the strongest predictor of whether an email exists.

Max subs Filters
Subscriber ceiling. `0` means none.

Recency (days) Filters
How far back an upload still counts as recent.

Min uploads Filters
How many uploads are needed inside that window for a channel to count as active.

Min avg views Filters
Optional floor on average views. `0` means off.

Ignore Shorts-only channels
Rejects channels that publish almost nothing but Shorts.

Skip channels seen before
Passes over anything already evaluated by any earlier run. Makes repeat runs fast, and is the usual reason a later run finds nothing.

Read video descriptions for emails
Adds a pass over recent video descriptions on top of the About page. Slower, and where a good share of emails actually live.

Find sponsor brands
Reads those same descriptions for sponsorships, and fills the `Sponsors` column.

Workers Engine
How many channels are evaluated at the same time. 4 is the ceiling on a home connection.

Max requests / sec Engine

A global speed limit across all workers. The engine also throttles itself below this whenever YouTube pushes back.

Depth per keyword Engine

How many videos are scanned for each keyword. The honest way to get a bigger list.

Discovery surfaces

Which parts of YouTube are searched: `Videos · relevance`, `Videos · recent` and `Channels`. Each one you switch on multiplies the number of search tasks.

Search tasks

Keywords × markets × surfaces. The hint line under the keyword box computes it live, and it is the best available estimate of how long a run will take.

ALONG THE TOP

direct · gateway · list

The connection mode, shown in the first pill. Your own address; a provider's rotating endpoint; or your own list of proxies.

req/s

The live request rate — how fast the app is actually going right now, which is often below your ceiling because it is backing off.

saved

How many channels are in the database altogether, across every run you have ever done.

idle · running · paused

What the engine is doing. `Pause` holds a run in place; `Stop` ends it and keeps everything found so far.

IN THE RESULTS TABLE

Average views

A *trimmed* mean of recent uploads — highest and lowest discarded — so one viral video cannot flatter an ordinary channel.

Score

0–100, and the default sort. Reach 32%, efficiency 26%, activity 15%, freshness 12%, momentum 9%, consistency 6%.

Momentum

The newest three uploads measured against the channel's own baseline. Above 1 means accelerating.

Views / sub

Average views divided by subscribers. 0.10 is healthy; 0.35 and up is exceptional. The number that separates an engaged audience from a large one.

Consistency

How predictable the view counts are. High consistency is what you are actually buying with a sponsorship.

Last upload

Days since the most recent video.

Sponsors

Brands found in recent descriptions, plus YouTube's own "includes paid promotion" disclosure.

IN SIMILAR CHANNELS AND WHO SPONSORED

Match

How close a neighbour is: co-viewing first, then comparability in size and country.

Why it ranked here

The reasoning behind a neighbour's position, in words rather than a number.

Also works with

How many other creators in your database a brand has sponsored, and their median size. A roster means a programme, a rate card, and somebody whose job it is to answer.

Proof

Links to the actual uploads a sponsorship was read from.

Brands to pitch next

Companies ranked by how likely they are to buy this creator, scored against the brand ledger.

Brand ledger

The record of every sponsorship the tool has ever read, across every creator. It fills as you search, and the recommendations sharpen with it.

WORDS THE APP USES WHEN IT REJECTS A CHANNEL

already seen · shorts-only · inactive

Found by an earlier run; publishes almost only Shorts; did not clear `Min uploads` inside `Recency (days)`.

too few subs · too many subs · avg views too low

Below `Min subs`; above `Max subs`; below `Min avg views`.

outside your markets · no country published

Published a country that is not in your markets; published no country at all while `Country scope` was strict.

subs unknown

No subscriber count could be tied to the channel itself, so it was dropped rather than guessed.

no feed

YouTube served no upload feed. Usually rate limiting rather than a dead channel.

IN THE SETTINGS FILE

PROXY_MODE · PROXY_GATEWAY

`direct`, `gateway` or `list`; and the full gateway URL including credentials.

Sticky session

How long one worker keeps a single exit address before rotating. 25 requests is a sensible default; `0` takes a new address every request, which is usually worse.

WORKERS · MAX_RPS · SEARCH_DEPTH

The `Engine` sliders' starting values.

MIN_SUBSCRIBERS · MAX_SUBSCRIBERS · RECENT_DAYS · MIN_RECENT_VIDEOS

The `Filters` starting values.

COUNTRY_FILTER

The `Country scope` default: `off`, `markets` or `strict`.

DATA_DIR · HOST · PORT

Where the database and exports live, and where the dashboard listens.

TERMINAL WORDS

Terminal · PowerShell

The window where you type commands. Terminal on a Mac, PowerShell on Windows.

`cd`

"Change directory" — moves the terminal into a folder. Everything you type afterwards happens there.

Virtual environment · `.venv`

A private folder holding this app's libraries only, so nothing it installs disturbs anything else. Your prompt shows `(.venv)` when it is active.

`pip`

Python's installer. `pip install -r requirements.txt` reads the app's list of parts and fetches them.

localhost · port

localhost means your own machine. The *port* — 8000 by default — is which door on it the dashboard is listening at.

Flag

A word added after a command to change how it starts, like `--port 9000` or `--check`.

Two commands are worth remembering above all others. `python app.py --check` proves the setup is sound, offline and in a second. `python app.py --diagnose @somechannel` walks one channel through every stage and shows you exactly where it fell out. Between them they answer most questions this manual does not.